ENTREGA 03 / PROJETO INTEGRADOR 3

Testes unitários
Cadastro catálogo e sacola

Entrega 3 - Parte 2

Projeto	ISAC Brechó - Bazar Solidário
Parceiro	ISAC, por meio do Porto Digital
Módulo	Três classes de serviço e suas classes de teste
Revisão	3.0 | 18 de outubro de 2026
GitHub	https://github.com/atlasaqui/bazar-solidario
Branch publicada	entrega/03-testes-unitarios-parte-2
Código publicado	https://github.com/atlasaqui/bazar-solidario/tree/entrega/03-testes-unitarios-parte-2

Sobre esta entrega

Nesta etapa, ampliamos os testes do cadastro para as regras de consulta ao catálogo e de uso da sacola. As três classes estão ligadas à interface do ISAC Brechó e possuem classes de teste próprias. A execução completa terminou com 34 testes aprovados.

Equipe

Victor William Monteiro da Rocha
Geovana Leticia
Silas Freitas
Mateus Ferreira
Willyanne Darline
Ruan Araujo
Suellen da Silva
Cynthia Melo

Resultado desta etapa

3 classes de serviço | 3 classes de teste | 34 testes aprovados | 0 falhas

Classes e integração

Evolução em relação à Parte 1

Critério	Parte 1	Parte 2
Classes de serviço	1 cadastro	3 cadastro, catálogo e sacola
Testes executados	14	34
Novos cenários	Cadastro e contrato HTTP	Mais 9 de catálogo e 11 de sacola

Classe	Responsabilidade	Testes
UsuarioService	Validação do cadastro e contrato HTTP.	7 + 7 regressões
CatalogoService	Busca, filtros, favoritos e ordenação.	9
SacolaService	Peças únicas, estado da sacola e subtotal.	11

Uso no aplicativo

Cadastro.tsx chama UsuarioService ao enviar o formulário. App.tsx usa CatalogoService na listagem das peças e SacolaService ao carregar, adicionar e remover itens e calcular o subtotal. As classes testadas são usadas pelo aplicativo, em vez de existir apenas para os testes.

Serviços	frontend/src/services/usuarios.ts frontend/src/services/CatalogoService.ts frontend/src/services/SacolaService.ts
Classes de teste	UsuarioServiceTest CatalogoServiceTest SacolaServiceTest
Local dos testes	frontend/tests/
Isolamento	Cada método usa uma nova instância da classe de teste. As chamadas HTTP são simuladas.

Testes do catálogo

CatalogoService / 9 testes

Cenário executado	Resultado
lista todas as peças por padrão	Aprovado
busca nome ignorando acentos, caixa e espaços externos	Aprovado
busca pelo brechó	Aprovado
combina categoria, tamanho, favoritos e texto	Aprovado
retorna vazio quando não há correspondência	Aprovado
favoritos vazios não exibem todo o catálogo	Aprovado
ordena preços crescentes e decrescentes	Aprovado
mantém curadoria após ordenar e não permite modificar o catálogo pela resposta	Aprovado
aceita catálogo vazio	Aprovado

Testes da sacola

SacolaService / 11 testes

Cenário executado	Resultado
adiciona uma peça sem modificar o estado anterior	Aprovado
não duplica uma peça única	Aprovado
rejeita peça inexistente	Aprovado
remove somente a peça escolhida	Aprovado
remover ID ausente mantém os itens válidos	Aprovado
restaura IDs válidos e descarta duplicatas e valores inválidos	Aprovado
recupera armazenamento vazio, corrompido ou não-array	Aprovado
calcula subtotal apenas de peças válidas, sem duplicatas	Aprovado
calcula valores monetários em centavos	Aprovado
sacola vazia tem subtotal zero	Aprovado
itens retornados não permitem alterar preços internos	Aprovado

Testes do cadastro

UsuarioService / 7 testes

Cenário executado	Resultado
valida campos antes de fazer qualquer requisição	Aprovado
rejeita confirmação diferente sem enviar dados	Aprovado
aceita limites de senha e nome e retorna apenas dados públicos	Aprovado
rejeita identificador inválido e perfil inesperado	Aprovado
trata HTTP 422	Aprovado
trata HTTP 503	Aprovado
trata HTTP 504	Aprovado

Contrato HTTP do cadastro / 7 testes

Cenário executado	Resultado
cadastro respeita o contrato existente e descarta a senha da resposta	Aprovado
falha de rede produz mensagem recuperável	Aprovado
erro HTTP 400 não expõe detalhes internos do backend	Aprovado
erro HTTP 409 não expõe detalhes internos do backend	Aprovado
erro HTTP 500 não expõe detalhes internos do backend	Aprovado
erro HTTP 502 não expõe detalhes internos do backend	Aprovado
resposta inválida não apresenta falso sucesso	Aprovado

Resultado e evidências

34	34	0	3
executados	aprovados	falhas	suítes

Data da execução	18/10/2026 às 09:19:50 (Fortaleza)
Ambiente	Node v22.23.2 | Node Test Runner
Comando	npm run test:evidencias, dentro de frontend
Estado final	34 aprovados; 0 falhas; 0 cancelados; 0 ignorados; código de saída 0.
Registro completo	docs/pi3/evidencias/testes.tap e resultado.json

Conferência no protótipo

Fluxo	Resultado observado
Catálogo	A busca TENIS encontrou Tênis casual mesmo sem acento.
Sacola	Camisa de linho e Bolsa caramelo somaram R$ 154,00.
Cadastro	Senhas diferentes produziram a mensagem de validação.

Os prints do protótipo são registros de conferência manual. Eles complementam os testes unitários, que verificam as classes diretamente. A imagem geral dos testes foi exportada por inteiro e também separada por classe para facilitar a leitura.

Referências da implementação

Repositório	https://github.com/atlasaqui/bazar-solidario
Branch da Parte 1	entrega/02-testes-unitarios-parte-1
Código da Parte 1	https://github.com/atlasaqui/bazar-solidario/tree/entrega/02-testes-unitarios-parte-1
Branch da Parte 2	entrega/03-testes-unitarios-parte-2
Código da Parte 2	https://github.com/atlasaqui/bazar-solidario/tree/entrega/03-testes-unitarios-parte-2
Evidências	docs/pi3/evidencias/testes.tap docs/pi3/evidencias/resultado.json

Conclusão

A segunda etapa amplia a implementação de uma para três classes de serviço. Os 14 testes do cadastro foram preservados, e os testes de catálogo e sacola acrescentaram 20 cenários. A execução completa terminou com 34 aprovações e nenhuma falha.

Escopo da verificação

Os resultados se referem ao front-end do ISAC Brechó. Os testes HTTP simulam a API, enquanto catálogo e sacola usam os dados demonstrativos do projeto. As classes Java do backend, pagamento, pedidos reais, chatbot e login não integram esta suíte.